Bingo de Halloween virtual

Documento del proyecto: descripción, alcance, requerimientos, fases y stack

Proyecto de uso privado para un grupo de 6 amigos. Vigencia: fin de semana de Halloween, con cierre del proyecto al terminar.

1. Descripción general

Aplicación web de bingo de Halloween pensada para jugar desde el celular. Cada jugador entra con su nombre y recibe un tablero 3×3 cuyas celdas son categorías de disfraces (por ejemplo, "Payaso" o "Policía"). Para completar una celda, el jugador saca o elige una foto, que se sube a UploadThing. Gana quien complete el tablero completo. Al finalizar, elige una foto final por celda y se genera un informe PDF con las horas de emisión y de la última foto, que el grupo usa para verificar el orden de llegada y decidir quién gana.

El proyecto no tiene login, base de datos ni moderación automática. La validación final del resultado la hace el grupo a partir del informe.

2. Objetivos

- Permitir que cada jugador complete su tablero 3×3 con fotos desde el celular.
- Garantizar que ninguna foto se pierda aunque falle la conexión.
- Generar un informe PDF con evidencia temporal para decidir el ganador.
- Entregar una versión funcional en pocos días, con costo cero de infraestructura.

3. Alcance

Incluye:

- Ingreso con nombre e identificador por dispositivo (UUID en localStorage).
- Tablero 3×3 determinístico a partir del UUID, con 9 categorías definidas por el organizador.
- Carga de múltiples fotos por celda (cámara o galería), vista previa inmediata y subida a UploadThing.
- Cola local de fotos pendientes en IndexedDB, con reintento manual y automático.
- Galería por celda, eliminación de fotos y contador de progreso.
- Elección de foto final por celda y generación del informe PDF.
- Respaldo copiable y restauración desde texto, para recuperar el tablero en otro dispositivo.

No incluye:

- Login, cuentas, contraseñas ni roles de administrador.
- Base de datos ni persistencia en servidor.
- Moderación automática de fotos, reconocimiento de disfraces o validación con IA.
- Tablero de posiciones, notificaciones, ZIP de fotos o compartir informe por link (quedan para una evolución futura).
- Soporte para más de 6 jugadores o uso público.

4. Usuarios

Jugador: amigo invitado que entra por link, completa su tablero y elige sus fotos finales. Puede ver las fotos de todos.

Organizador: define las 9 categorías, comparte el link, y junto con el grupo valida las fotos y determina el ganador a partir del informe.

5. Reglas del juego

- Cada jugador tiene un tablero 3×3 fijo, generado a partir de su identificador.
- Cada celda admite varias fotos. Al finalizar se elige una sola por celda.
- Ganar es completar las 9 celdas (cada una con al menos una foto).
- La hora registrada es la del reloj del celular. Para este juego entre amigos es suficiente.
- Si hay disputas sobre una foto, el grupo decide en conjunto.

6. Requerimientos funcionales

Prioridad: **Must** = imprescindible para el MVP; **Should** = importante pero postergable; **Could** = deseable si hay tiempo. Fase: número de fase del plan (ver sección 8).

ID	Nombre	Descripción	Prioridad	Fase
RF-01	Ingreso con nombre	El usuario ingresa un nombre al entrar, sin login ni contraseña.	Must	2
RF-02	Identificador único por dispositivo	Al primer ingreso se genera un UUID guardado en localStorage que identifica al jugador y sus datos.	Must	2
RF-03	Código de recuperación	Se muestra el UUID (o código) con aviso para guardarlo, y campo para recuperar el tablero desde otro dispositivo.	Must	2
RF-04	Tablero 3×3 determinístico	El tablero se genera a partir del UUID como semilla, siempre igual para el mismo jugador.	Must	1
RF-05	Categorías del tablero	Cada celda corresponde a una categoría de disfraz (9 en total, definidas por el organizador).	Must	1
RF-06	Agregar foto a una celda	Al tocar una celda se abre el selector nativo (cámara o galería).	Must	3
RF-07	Múltiples fotos por celda	Agregar una foto no reemplaza las anteriores.	Must	3
RF-08	Vista previa inmediata	La foto aparece al instante, antes de terminar la subida (optimistic UI).	Must	3
RF-09	Subida a UploadThing	Cada foto se intenta subir a UploadThing al agregarla.	Must	3
RF-10	Estado de subida	Cada foto muestra si está pendiente o subida.	Must	3
RF-11	Reintento manual	Botón visible para reintentar la subida de las fotos pendientes.	Must	3
RF-12	Reintento automático	Al recuperar la conexión se reintentan las fotos pendientes.	Should	3
RF-13	Registro de hora de captura	Cada foto guarda fecha y hora en que fue elegida.	Must	3
RF-14	Eliminar foto	El usuario puede borrar una foto equivocada.	Must	4
RF-15	Detección de tablero completo	Con las 9 celdas con al menos una foto aparece la opción de finalizar.	Must	5
RF-16	Elección de foto final	Al finalizar, el usuario elige una foto por celda entre las que sacó.	Must	5
RF-17	Cambio de foto elegida	Se puede cambiar la foto elegida de una celda antes de generar el informe.	Should	5
RF-18	Generación de informe PDF	PDF con el tablero, la foto elegida por celda y su hora.	Must	6
RF-19	Horas en el informe	El PDF muestra la hora de emisión y la hora de la última foto subida.	Must	6
RF-20	Subida del PDF	El PDF se sube a UploadThing en una carpeta separada de las fotos.	Should	6

ID	Nombre	Descripción	Prioridad	Fase
RF-21	Respaldo copiable	Genera un texto con UUID y URLs de las fotos para copiar (ej. WhatsApp).	Should	3
RF-22	Restauración desde respaldo	Campo que lee un respaldo pegado y reconstruye tablero y fotos.	Should	3
RF-23	Alta en pantalla de inicio	Sugerir instalar la página en el celular para reducir pérdida de datos en iOS.	Should	7
RF-24	Acceso por link	Cualquiera con el link puede entrar y jugar, sin registrarse.	Must	2
RF-25	Ver fotos de todos	Las fotos del tablero son visibles para quien tenga el link.	Must	4
RF-26	Bingo por tablero completo	Ganar es completar el tablero 3×3; la validación final la hace el grupo.	Must	5
RF-27	Galería a pantalla completa	Al tocar una foto se abre a pantalla completa, con zoom y navegación entre las fotos de la celda.	Should	4
RF-28	Contador de progreso	Indicador "X/9 celdas completadas" visible en todo momento.	Should	4
RF-29	Deshacer al borrar foto	Tras eliminar una foto, aviso "Deshacer" durante unos segundos.	Could	4

7. Requerimientos no funcionales y técnicos

ID	Nombre	Descripción	Prioridad	Fase
NF-01	Mobile-first	La interfaz está pensada primero para celulares.	Must	Transv.
NF-02	Compresión de imágenes	Reducir en el cliente (lado máx. ~1600 px, JPEG 0.8) antes de subir.	Must	3
NF-03	Límite de tamaño	Cada foto subida no supera 8 MB.	Must	3
NF-04	Persistencia local	Las fotos pendientes se conservan en IndexedDB hasta confirmar la subida.	Must	3
NF-05	Persistencia del navegador	Solicitar almacenamiento persistente para que el navegador no borre datos.	Should	3
NF-06	Subida directa del cliente	Las fotos se suben desde el navegador a UploadThing, sin pasar por la API propia (límite de body en Vercel).	Must	3
NF-07	Sin base de datos	No hay persistencia en servidor; el estado vive en el cliente y en UploadThing.	Must	Transv.
NF-08	Seguridad mínima	Sin login ni validaciones complejas; uso previsto para un grupo cerrado de 6 personas.	Must	Transv.
NF-09	Disponibilidad temporal	El proyecto se cierra después del fin de semana; sin mantenimiento a largo plazo.	Must	Transv.
NF-10	Tolerancia a conexión inestable	Una caída de red no hace perder fotos ni rompe la experiencia.	Must	3
NF-11	Rendimiento de la UI	La vista previa aparece sin esperar la subida.	Must	3
NF-12	Estilo y maquetado	CSS Modules por componente y variables CSS para la paleta de Halloween.	Should	1
NF-13	Stack	Next.js (App Router), UploadThing, Vercel y GitHub.	Must	1
NF-14	Acceso sin conexión	Con service worker, la app abre y muestra tablero y fotos guardadas sin internet. Subir fotos sigue requiriendo conexión.	Should	7
T-01	Generación de PDF en servidor	PDF con @react-pdf/renderer en una route handler (/api/informe).	Must	6
T-02	Ruta de subida de UploadThing	Endpoint con maxFileCount 1 y tipo imagen, en app/api/uploadthing.	Must	3

ID	Nombre	Descripción	Prioridad	Fase
T-03	Modelo de datos local	IndexedDB con las fotos (blob, estado, URL, timestamps) y el jugador en localStorage.	Must	3

8. Fases del proyecto

Fase	Nombre	Objetivo	Requerimientos	Criterio de salida
0	Preparación	Dejar el entorno listo y desplegado	NF-13, NF-07, NF-08, NF-09	Next.js en Vercel, UploadThing configurado, repo en GitHub
1	Tablero estático	Grilla 3×3 con las categorías	RF-04, RF-05, NF-01, NF-12	Se ve bien en celular y sale igual para el mismo UUID
2	Jugador y acceso	Identificar al jugador sin login	RF-01, RF-02, RF-03, RF-24	Entrar, cerrar, volver a entrar y seguir con el mismo tablero
3	Fotos y subida	Agregar, subir y manejar fallas	RF-06 a RF-13, RF-21, RF-22, NF-02 a NF-06, NF-10, NF-11, T-02, T-03	Foto al instante; si falla queda pendiente con botón de reintento
4	Galería por celda	Ver, borrar y revisar fotos	RF-14, RF-25, RF-27, RF-28, RF-29	Ver fotos de una celda, borrar una y deshacer
5	Finalizar y elegir	Elegir la foto final de cada celda	RF-15, RF-16, RF-17, RF-26	Con 9/9 aparece Finalizar y se elige una foto por celda
6	Informe PDF	Generar y subir el informe	RF-18, RF-19, RF-20, T-01	PDF con las 9 fotos, hora de emisión y última foto
7	Pulido	Experiencia y pruebas en dispositivos reales	RF-23, NF-14	Probado en iPhone y Android, con modo sin conexión básico

9. MVP

El MVP permite jugar de principio a fin: cada amigo entra, completa su tablero, elige sus fotos finales y el grupo recibe un informe.

Incluye: todos los requerimientos Must de las fases 0 a 6, más RF-21 (respaldo copiable), recomendado por ser la única protección contra pérdida de datos en iOS o al cambiar de celular.

Queda fuera: RF-12, RF-17, RF-20, RF-22, RF-23, RF-27, RF-28, RF-29, NF-05 y NF-14. Si el tiempo alcanza, el orden sugerido es RF-28 y RF-27 primero.

Definición de MVP listo: desde el link de Vercel, cada uno de los 6 amigos entra con su nombre, completa las 9 celdas desde el celular, cierra la página y vuelve a entrar sin perder nada, elige una foto final por celda y descarga un PDF con las horas. Si el flujo funciona en un iPhone y en un Android, el MVP está terminado.

10. Stack tecnológico

Área	Tecnología / decisión
Framework	Next.js con App Router y Route Handlers
Estilos	CSS Modules por componente y variables CSS (paleta Halloween)
Estado	useState / useReducer; sin Redux
Subida de archivos	UploadThing (SDK de React y de Next.js). Subida directa desde el cliente
Almacenamiento local	localStorage (UUID, nombre) e IndexedDB (fotos, blobs, estado)
Compresión	Canvas en el navegador (lado máx. ~1600 px, JPEG 0.8)
Informe PDF	@react-pdf/renderer en /api/informe; subida con UTApi
Tablero	Semilla a partir del UUID para generar el tablero de forma reproducible
Offline (fase 7)	Service worker con Serwist o next-pwa
Despliegue	Vercel, con el repositorio en GitHub

11. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Pérdida del UUID o de datos locales (borrar datos, cambiar de celular, iOS)	Alta	Respaldo copiable (RF-21), código de recuperación (RF-03), pedir almacenamiento persistente (NF-05)
Fotos que no suben por mala conexión	Media	Cola local en IndexedDB, reintento manual y automático, estado visible
Fotos pesadas que ralentizan la subida	Media	Compresión en el cliente (NF-02) y límite de 8 MB (NF-03)
Límite de body de Vercel al subir fotos	Baja	Subida directa a UploadThing, sin pasar por la API (NF-06)
Hora poco confiable (reloj del celular)	Baja	Aceptado para el contexto; el grupo valida el resultado
Suplantación por usar el mismo nombre o UUID	Baja	Aceptado: grupo cerrado de 6 personas
Fotos de personas que no consintieron	Media	Link solo para el grupo; cada uno decide qué sube
Cierre del proyecto y pérdida de fotos tras el fin de semana	Media	Informe PDF con URLs como respaldo; descargar el PDF antes del cierre

12. Decisiones pendientes

- Lista final de las 9 categorías del tablero.
- Nombres de los 6 jugadores y cómo se comparte el link.
- Si el tablero queda bloqueado para subir fotos al finalizar (recomendado: sí).
- Fecha y hora de cierre de la página.
- Si el informe PDF se sube a UploadThing o solo se descarga (RF-20 es Should).